

DAY 13 — WINDOW FUNCTIONS

Today is one of the **most important SQL days** in your roadmap.

You've learned:

Day 8 → JOIN

Day 9 → LEFT / RIGHT / FULL JOIN

Day 10 → Multiple JOINS

Day 11 → Subqueries

Day 12 → CTEs

Today:

Window Functions

Window functions are heavily used in:

- Data Engineering
- Analytics
- Reporting
- ETL/ELT pipelines
- Ranking
- Deduplication
- Top-N problems
- Running totals
- Time-series analysis
- RAG evaluation and monitoring

The most important functions today are:

ROW_NUMBER()

RANK()

DENSE_RANK()

SUM() OVER()

AVG() OVER()

COUNT() OVER()

LAG()

LEAD()

1. The Big Difference: GROUP BY vs Window Functions

You already know:

```
GROUP BY
```

Suppose:

documents

```
user_id | page_count
```

```
-----|-----
```

```
1 | 20
```

```
1 | 40
```

```
1 | 60
```

```
2 | 30
```

```
2 | 50
```

Using:

```
SELECT
  user_id,
  SUM(page_count)
FROM documents
GROUP BY user_id;
```

You get:

```
user_id | total
```

```
-----|-----
```

```
1 | 120
```

```
2 | 80
```

The original rows disappear.

Window function

Now:

```
SELECT
  user_id,
```

```

page_count,

SUM(page_count) OVER (
  PARTITION BY user_id

) AS user_total_pages
FROM documents;

```

Result:

user_id | page_count | user_total_pages

-----|-----|-----

1 | 20 | 120

1 | 40 | 120

1 | 60 | 120

2 | 30 | 80

2 | 50 | 80

This is the fundamental idea:

GROUP BY collapses rows. Window functions calculate across rows while keeping the original rows.

2. Window Function Syntax

General syntax:

FUNCTION(...) OVER (

PARTITION BY ...

ORDER BY ...

)

Example:

```

SUM(page_count) OVER (
  PARTITION BY user_id

)

```

There are three important pieces:

SUM()

↓

what calculation?

OVER()



window calculation

PARTITION BY



which rows belong together?

3. OVER()

The simplest window function:

```
SELECT
  id,
  title,
  page_count,
  COUNT(*) OVER () AS total_documents
FROM documents;
```

Suppose there are 10 documents.

Every row gets:

total_documents = 10

Example:

id	title	total_documents
101	RAG Fundamentals	10
102	PostgreSQL Guide	10
103	Python Basics	10

Unlike:

```
SELECT COUNT(*)
FROM documents;
```

which returns only one row.

4. PARTITION BY

Think of:

PARTITION BY

as:

Divide rows into groups for the window calculation.

Example:

```
SELECT
  user_id,
  title,
  page_count,
  SUM(page_count) OVER (
    PARTITION BY user_id
  ) AS user_total_pages
FROM documents;
```

Partition:

User 1

├— Document A

├— Document B

└— Document C

User 2

├— Document D

└— Document E

The SUM happens separately for each user.

5. GROUP BY vs PARTITION BY

This is an important interview question.

GROUP BY

```
SELECT
  user_id,
```

```
SUM(page_count)
FROM documents
GROUP BY user_id;
```

Result:

1 | 120

2 | 80

Rows are collapsed.

Window function

```
SELECT
user_id,

title,

page_count,

SUM(page_count) OVER (
PARTITION BY user_id

) AS total_pages
FROM documents;
```

Result:

1 | A | 20 | 120

1 | B | 40 | 120

1 | C | 60 | 120

2 | D | 30 | 80

2 | E | 50 | 80

Original rows remain.

6. ROW_NUMBER()

One of the most important window functions.

Syntax:

```
ROW_NUMBER() OVER (
ORDER BY ...
)
```

Example:

```
SELECT
  id,
  title,
  page_count,
  ROW_NUMBER() OVER (
    ORDER BY page_count DESC
  ) AS row_number
FROM documents;
```

Result:

title	page_count	row_number
LLM Fundamentals	60	1
Embeddings Guide	55	2
Vector Databases	50	3
PostgreSQL Guide	45	4

Every row receives a unique number.

7. ROW_NUMBER With PARTITION BY

Now:

Rank documents separately for each user.

```
SELECT
  user_id,
  title,
  page_count,
  ROW_NUMBER() OVER (
    PARTITION BY user_id
    ORDER BY page_count DESC
  ) AS document_rank
FROM documents;
```

Conceptually:

User 1

Document A → 60 → 1

Document B → 40 → 2

Document C → 20 → 3

User 2

Document D → 50 → 1

Document E → 30 → 2

The numbering restarts for each user.

8. Real Data Engineering Use Case: Top Document Per User

Suppose you want:

The largest document uploaded by every user.

You cannot simply do:

```
ORDER BY page_count DESC  
LIMIT 1
```

because that gives only one document globally.

Use ROW_NUMBER():

```
WITH ranked_documents AS (  
  SELECT  
    id,  
    user_id,  
    title,  
    page_count,  
    ROW_NUMBER() OVER (  
      PARTITION BY user_id  
      ORDER BY page_count DESC  
    ) AS rn  
  FROM documents  
)  
SELECT  
  id,  
  user_id,
```

title,

page_count

FROM ranked_documents

WHERE rn = 1;

This pattern is extremely important.

9. Why We Need a CTE Here

You cannot normally write:

```
WHERE ROW_NUMBER() OVER (...) = 1
```

in the same SELECT level.

Instead:

Step 1

↓

calculate row number

Step 2

↓

filter row number

So:

```
WITH ranked_documents AS (...)
```

```
SELECT *
```

```
FROM ranked_documents
```

```
WHERE rn = 1;
```

This combines:

Day 12 → CTE

+

Day 13 → Window Function

10. RANK()

Now let's look at ties.

Suppose:

page_count

100

100

80

70

Using:

```
RANK() OVER (  
    ORDER BY page_count DESC  
)
```

Result:

page_count | rank

-----|-----

100 | 1

100 | 1

80 | 3

70 | 4

Notice:

1

1

3

4

Rank 2 is skipped.

11. DENSE_RANK()**Now:**

```
DENSE_RANK() OVER (  
    ORDER BY page_count DESC  
)
```

Result:

page_count | dense_rank

-----|-----

100 | 1

100 | 1

80 | 2

70 | 3

No gap.

12. ROW_NUMBER vs RANK vs DENSE_RANK

Memorize this table:

Function	Ties	Example
ROW_NUMBER()	unique numbers	1, 2, 3, 4
RANK()	same rank, gaps	1, 1, 3, 4
DENSE_RANK()	same rank, no gaps	1, 1, 2, 3

Example:

Scores:

100

100

90

80

Results:

ROW_NUMBER

1

2

3

4

RANK

1

1

3

4

DENSE_RANK

1

1

2

3

This is a very common interview question.

13. When Should You Use Which?

ROW_NUMBER()

Use when you need exactly one ordering position per row.

Common use:

deduplication

top 1 per group

latest record per customer

RANK()

Use when tied values should share the same rank and ranking gaps should exist.

Common:

competition rankings

leaderboards

DENSE_RANK()

Use when tied values should share rank but you don't want gaps.

14. Ranking Per User

```
SELECT
user_id,

title,

page_count,


ROW_NUMBER() OVER (
PARTITION BY user_id

ORDER BY page_count DESC
) AS row_num,

RANK() OVER (
PARTITION BY user_id

ORDER BY page_count DESC
) AS rank_num,

DENSE_RANK() OVER (
PARTITION BY user_id

ORDER BY page_count DESC
) AS dense_rank_num
```

FROM documents;

You can calculate multiple window functions in one query.

15. Window ORDER BY vs Query ORDER BY

Very important.

Consider:

```
SELECT
title,

page_count,

ROW_NUMBER() OVER (
ORDER BY page_count DESC
) AS rn
FROM documents;
```

The window's:

ORDER BY

controls how the row number is calculated.

It does **not necessarily guarantee the final output order**.

If you want the final result sorted:

SELECT

title,

page_count,

ROW_NUMBER() OVER (

ORDER BY page_count DESC

) AS rn

FROM documents

ORDER BY page_count DESC;

You can have both.

16. Running Total

Window functions aren't only for ranking.

You can calculate a running total.

Suppose documents have:

page_count

20

30

40

50

Query:

SELECT

title,

page_count,

SUM(page_count) OVER (

ORDER BY id

ROWS BETWEEN UNBOUNDED PRECEDING

```
AND CURRENT ROW  
) AS running_pages  
FROM documents;
```

Conceptually:

20

20 + 30 = 50

20 + 30 + 40 = 90

20 + 30 + 40 + 50 = 140

Result:

	title	page_count	running_pages
A		20	20
B		30	50
C		40	90
D		50	140

17. Simpler Running Total Syntax

PostgreSQL often allows:

```
SELECT  
title,  
  
page_count,  
  
SUM(page_count) OVER (  
    ORDER BY id  
) AS running_pages  
FROM documents;
```

For many cases this works as expected.

But understanding the explicit frame:

ROWS BETWEEN UNBOUNDED PRECEDING

```
AND CURRENT ROW
```

is valuable because window frames matter when ordering values contain duplicates.

18. Running Total Per User

Now:

```
SELECT
  user_id,
  title,
  page_count,
  SUM(page_count) OVER (
    PARTITION BY user_id
    ORDER BY id
  ) AS running_user_pages
FROM documents;
```

Conceptually:

User 1

20 → 20

40 → 60

60 → 120

User 2

30 → 30

50 → 80

The running total resets for each user.

19. Window AVG

You can also calculate averages without losing rows.

```
SELECT
  user_id,
  title,
  page_count,
```

```

AVG(page_count) OVER (
PARTITION BY user_id

) AS user_average_pages
FROM documents;

```

Result:

	user_id	title	page_count	user_average_pages
	1	A	20	40
	1	B	40	40
	1	C	60	40
	2	D	30	40
	2	E	50	40

Now each document can be compared against its user's average.

20. Above-Average Documents Using Window Function

Yesterday we used a correlated subquery.

Today we can write:

```

WITH document_stats AS (
  SELECT
    id,

    user_id,

    title,

    page_count,

    AVG(page_count) OVER (
      PARTITION BY user_id

    ) AS user_average_pages
  FROM documents
)
SELECT
  id,

  user_id,

  title,

  page_count,

  user_average_pages

```

```
FROM document_stats
WHERE page_count > user_average_pages;
This is much cleaner.
```

21. COUNT() OVER()

You can calculate counts while preserving rows.

```
SELECT
user_id,

title,

COUNT(*) OVER (
PARTITION BY user_id

) AS documents_for_user
FROM documents;
```

If user 1 has 3 documents:

User 1 | Document A | 3

User 1 | Document B | 3

User 1 | Document C | 3

Again:

```
GROUP BY
→ collapses rows
```

```
COUNT() OVER()
→ keeps rows
```

22. Percentage of Total

A very useful analytics pattern.

```
SELECT
title,

page_count,
```

```
ROUND(
100.0 * page_count
```

```
    / SUM(page_count) OVER (),  
    2  
    ) AS percentage_of_total_pages  
FROM documents;
```

Suppose:

total pages = 1000

A document with 100 pages gets:

10.00%

This is useful for:

- contribution analysis
- resource distribution
- storage analysis
- traffic analysis
- usage reports

23. Percentage Within Each User

Now:

```
SELECT  
user_id,  
title,  
page_count,
```

ROUND(

```
    100.0 * page_count  
    / SUM(page_count) OVER (  
        PARTITION BY user_id  
    ),  
    2  
    ) AS percentage_of_user_pages  
FROM documents;
```

This answers:

What percentage of this user's total document pages belongs to this document?

24. LAG()

Now we move into time-series style analysis.

LAG() lets you access a previous row.

Example:

```
SELECT
id,
title,
page_count,
LAG(page_count) OVER (
ORDER BY id
) AS previous_page_count
FROM documents;
```

Result:

	title	page_count	previous_page_count
A		20	NULL
B		40	20
C		60	40
D		30	60

For the first row there is no previous row:

NULL

25. Why Is LAG Useful?

Suppose you're tracking daily:

documents_uploaded

You might want:

today

yesterday

difference

percentage change

LAG() gives you yesterday's value.

Example:

```
SELECT
upload_date,
document_count,
LAG(document_count) OVER (
ORDER BY upload_date
) AS previous_day_count
FROM daily_document_stats;
```

Then you can calculate:

$\text{document_count} - \text{previous_day_count}$

26. LAG Per User

```
SELECT
user_id,
title,
page_count,
LAG(page_count) OVER (
PARTITION BY user_id
ORDER BY id
) AS previous_document_pages
FROM documents;
```

Now each user has their own sequence.

27. LEAD()

LEAD() is the opposite.

It looks at the next row.

```
SELECT
id,
title,
page_count,
```

```
LEAD(page_count) OVER (  
  ORDER BY id  
) AS next_page_count  
FROM documents;
```

Conceptually:

LAG

← previous

current

LEAD

→ next

28. LAG + Difference

Example:

```
SELECT  
  id,  
  title,  
  page_count,  
  page_count  
    - LAG(page_count) OVER (  
      ORDER BY id  
    ) AS page_difference  
FROM documents;
```

Result conceptually:

20 → NULL

40 → 20

60 → 20

30 → -30

This is a very useful analytical pattern.

29. FIRST_VALUE()

PostgreSQL also supports:

FIRST_VALUE()

Example:

```
SELECT
  user_id,
  title,
  page_count,
  FIRST_VALUE(page_count) OVER (
    PARTITION BY user_id
    ORDER BY page_count DESC
  ) AS largest_document_pages
FROM documents;
```

Every document belonging to the user can now see:

largest document size for this user

30. LAST_VALUE() — Important Trap

LAST_VALUE() can be confusing because of window frames.

Don't rush to memorize it.

For example:

```
LAST_VALUE(page_count) OVER (
  PARTITION BY user_id
  ORDER BY page_count
)
```

may return the current row rather than the final row you expect because of the default frame.

To explicitly get the final value, you often need a frame such as:

ROWS BETWEEN UNBOUNDED PRECEDING

```
AND UNBOUNDED FOLLOWING
```

For now, focus on:

ROW_NUMBER

RANK

DENSE_RANK

SUM

AVG

COUNT

LAG

LEAD

31. Top N Per Group

This is one of the most important practical SQL problems.

Question:

Get the top 2 largest documents for every user.

Use:

```
WITH ranked_documents AS (  
  SELECT  
    id,  
  
    user_id,  
  
    title,  
  
    page_count,  
  
    ROW_NUMBER() OVER (  
      PARTITION BY user_id  
  
      ORDER BY page_count DESC  
    ) AS rn  
  FROM documents  
)  
SELECT  
  id,  
  
  user_id,
```

title,

page_count

FROM ranked_documents

WHERE rn <= 2

ORDER BY user_id, page_count DESC;

This pattern is extremely important.

32. Why LIMIT Doesn't Work

This:

ORDER BY page_count DESC

LIMIT 2

means:

Top 2 globally.

But we want:

Top 2 **for each user**.

That's why we need:

PARTITION BY user_id

and:

ROW_NUMBER()

33. Deduplication With ROW_NUMBER

A very common Data Engineering problem:

Suppose a table contains multiple records for the same logical entity.

Example:

email | updated_at

-----+-----

a@x.com | 2026-09-01

a@x.com | 2026-09-10

a@x.com | 2026-09-20

We want the latest record.

```

WITH ranked_users AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY email
      ORDER BY updated_at DESC
    ) AS rn
  FROM users_staging
)
SELECT *
FROM ranked_users
WHERE rn = 1;

```

This is one of the most common real-world uses of ROW_NUMBER().

34. ROW_NUMBER for RAG Deduplication

Imagine an ingestion table:

document_id

version

created_at

status

A document may have multiple ingestion attempts.

You want the latest attempt:

```

WITH ranked_ingestion AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY document_id
      ORDER BY created_at DESC
    ) AS rn
  FROM document_ingestion
)
SELECT *
FROM ranked_ingestion
WHERE rn = 1;

```

This gives one current record per document.

35. Window Functions + CASE

You can combine window calculations with CASE.

Example:

```
WITH document_stats AS (  
  SELECT  
    id,  
  
    user_id,  
  
    title,  
  
    page_count,  
  
    AVG(page_count) OVER (  
      PARTITION BY user_id  
  
    ) AS user_average  
  FROM documents  
)  
SELECT  
  id,  
  
  user_id,  
  
  title,  
  
  page_count,  
  
  user_average,  
  
  CASE  
    WHEN page_count > user_average  
      THEN 'Above Average'  
    WHEN page_count < user_average  
      THEN 'Below Average'  
    ELSE 'Average'  
  END AS size_category  
FROM document_stats;
```

This is a strong combination of:

Window Function

+

CTE

+

CASE

36. Window Functions and JOINS

You can use window functions after joining.

Example:

Rank documents within each country.

```
SELECT
  u.country,
  d.title,
  d.page_count,
  ROW_NUMBER() OVER (
    PARTITION BY u.country
    ORDER BY d.page_count DESC
  ) AS country_rank
FROM users u
JOIN documents d
  ON u.id = d.user_id;
```

Now ranking happens within:

India

USA

UK

Canada

separately.

37. Window Function Processing Mental Model

Consider:

```
SELECT
  title,
  page_count,
  ROW_NUMBER() OVER (
    ORDER BY page_count DESC
  ) AS rn
FROM documents
```

WHERE status = 'processed';

Conceptually:

documents

↓

WHERE status = processed

↓

remaining rows

↓

window function

↓

ROW_NUMBER

↓

SELECT

This is important.

The window function operates on the rows that survive the earlier filtering stage.

38. Window Functions vs GROUP BY

Remember this forever:

GROUP BY

Many rows

↓

One row per group

Window Function

Many rows

↓

Many rows

+

calculation across related rows

Example:

GROUP BY user

→ 1 row per user

```
SUM() OVER(PARTITION BY user)
```

→ every document row remains

39. Window Functions vs Subqueries

Yesterday:

```
WHERE page_count > (  
  SELECT AVG(...)  
)
```

Today:

```
AVG(page_count) OVER (  
  PARTITION BY user_id  
)
```

Both can solve related problems.

Window functions often make row-level analytical comparisons much easier because the aggregate is attached directly to each row.

40. Common Mistakes

Mistake 1 — Forgetting PARTITION BY

You want rank per user:

```
ROW_NUMBER() OVER (  
  ORDER BY page_count DESC  
)
```

This ranks globally.

You need:

```
ROW_NUMBER() OVER (  
  PARTITION BY user_id  
  
  ORDER BY page_count DESC  
)
```

Mistake 2 — Confusing RANK and ROW_NUMBER

If ties should share rank:

RANK

If every row needs a unique number:

ROW_NUMBER

Mistake 3 — Using LIMIT for Top-N Per Group

```
LIMIT 3
```

gives 3 globally.

Use window functions for:

3 per user

3 per country

3 per category

Mistake 4 — Filtering Window Functions in WHERE

This is invalid:

```
SELECT
*,
    ROW_NUMBER() OVER (
        ORDER BY page_count DESC
    ) AS rn
FROM documents
WHERE rn <= 3;
```

Why?

Because rn is generated by the SELECT/window stage.

Use a CTE:

```
WITH ranked AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            ORDER BY page_count DESC
        ) AS rn
    FROM documents
)
```

```
SELECT *  
FROM ranked  
WHERE rn <= 3;
```

41. Debugging Challenge #1

Developer wants:

Top 2 documents per user.

They write:

```
SELECT  
id,  
user_id,  
title,  
page_count,  
  
    ROW_NUMBER() OVER (  
        ORDER BY page_count DESC  
    ) AS rn  
FROM documents  
WHERE rn <= 2;
```

Find **both problems**.

42. Debugging Challenge #2

Developer writes:

```
ROW_NUMBER() OVER (  
PARTITION BY user_id  
  
    ORDER BY page_count DESC  
)
```

but gets:

User 1 → 1

User 1 → 2

User 2 → 3

User 2 → 4

instead of:

User 1 → 1

User 1 → 2

User 2 → 1

User 2 → 2

Is the window function actually wrong?

What else might be happening?

43. Debugging Challenge #3

Developer wants ranking where ties share the same rank.

They use:

```
ROW_NUMBER() OVER (  
    ORDER BY page_count DESC  
)
```

Example:

100

100

90

They get:

1

2

3

What function should they use instead?

And what would DENSE_RANK() produce?

44. SQL Exercises

Use:

users

documents

document_chunks

embeddings

Q1

Return every document with:

document_id

title

page_count

row_number

ordered by:

page_count DESC

Use ROW_NUMBER().

Q2

Rank documents by page count **within each user**.

Return:

user_id

title

page_count

document_rank

Q3

Return the largest document for every user.

Use:

ROW_NUMBER()

CTE

Q4

Return the top 2 documents for every user.

Q5

Calculate each user's total pages while keeping every document row.

Use:

```
SUM(page_count) OVER (...)
```

Q6

Calculate each user's average document size while keeping every document row.

Use:

```
AVG(page_count) OVER (...)
```

Q7

Return every document and the number of documents belonging to that same user.

Use:

```
COUNT(*) OVER (  
  PARTITION BY user_id  
)
```

45. Intermediate Exercises

Q8

For each document, return:

document_id

title

page_count

user_average_pages

Then return only documents whose page count is above their user's average.

Use a CTE.

Q9

Return the top 3 documents per country.

You'll need:

users

JOIN

documents

ROW_NUMBER

PARTITION BY country

Q10

Rank users by their total document pages.

First calculate:

user_id

total_pages

Then rank users globally.

Return:

user_id

total_pages

rank

Q11

Rank users by total document pages, but users with equal totals should share the same rank.

Use:

RANK()

Q12**Repeat Q11 using:**

DENSE_RANK()

Compare the results.

Q13

Calculate a running total of pages ordered by document ID.

Return:

document_id

page_count

running_total

Q14

Calculate a running total **per user**.

Return:

user_id

document_id

page_count

running_user_total

Q15

Use LAG() to show:

document_id

page_count

previous_document_page_count

ordered by document ID.

Q16

Calculate:

current_page_count - previous_page_count

using LAG().

46. Engineering Challenge — RAG Top Documents

Your RAG dashboard needs:

The top 2 largest documents for every user.

Return:

user_id

user_name

document_id

document_title

page_count

rank

Requirements:

- Join users and documents.
- Rank documents within each user.
- Use ROW_NUMBER().
- Use PARTITION BY.
- Use ORDER BY page_count DESC.
- Return only the top 2 per user.
- Use a CTE.

47. Engineering Challenge 2 — RAG Pipeline Ranking

Suppose you have:

documents

document_chunks

embeddings

Build a query returning each document:

document_id

document_title

chunk_count

embedding_count

embedding_success_rate

rank_within_status

Where:

embedding_success_rate =

$\text{ready_embeddings} / \text{total_embeddings} \times 100$

Then rank documents by success rate within each document status.

Use:

CTE

LEFT JOIN

GROUP BY

COUNT

CASE

window function

Be careful with:

NULL

division by zero

48. Interview Questions

Answer in your own words.

Q17

What is a window function?

Q18

What's the difference between:

GROUP BY

and:

PARTITION BY

?

Q19

What's the difference between:

ROW_NUMBER

RANK

DENSE_RANK

?

Q20

Why is ROW_NUMBER() useful for deduplication?

Q21

Why can't you normally use a window-function alias directly in WHERE at the same SELECT level?

Q22

Why is PARTITION BY important for Top-N-per-group problems?

Q23

What does LAG() do?

Q24

What does LEAD() do?

Q25

Give one real Data Engineering use case for window functions.

49. Explanation Challenge

Explain this query line by line:

```
WITH ranked_documents AS (  
  SELECT  
    d.id AS document_id,  
    d.user_id,  
    d.title,  
    d.page_count,  
    ROW_NUMBER() OVER (  
      PARTITION BY d.user_id
```

```

        ORDER BY d.page_count DESC, d.id
    ) AS rn
FROM documents d
)
SELECT
document_id,
user_id,
title,
page_count,
rn
FROM ranked_documents
WHERE rn <= 2
ORDER BY user_id, rn;

```

Your explanation must cover:

1. Why a CTE is used.
2. What PARTITION BY d.user_id does.
3. What ORDER BY d.page_count DESC does inside the window.
4. Why d.id is included as a second ordering column.
5. Why ROW_NUMBER() is used.
6. Why WHERE rn <= 2 is outside the CTE.
7. Why this returns up to 2 documents per user.
8. How ties are handled.
9. What would happen if RANK() replaced ROW_NUMBER().

50. Data Engineering Thinking

Imagine:

documents = 500 million

You need:

Latest record per document ID from a staging table.

You write:

```
WITH ranked AS (
```

```

SELECT
*,

ROW_NUMBER() OVER (
PARTITION BY document_id

ORDER BY updated_at DESC
) AS rn
FROM document_staging
)
SELECT *
FROM ranked
WHERE rn = 1;

```

Think about:

Question 1

Why can this be expensive on 500 million rows?

Question 2

What does PostgreSQL need to conceptually do to determine row numbers?

Question 3

Why is:

PARTITION BY document_id
ORDER BY updated_at DESC
important?

Question 4

What kind of index might help?

Question 5

What would you use later to investigate whether the query is actually efficient?

Hint:

EXPLAIN

You'll learn that on **Day 20**.

51. Short Test

Q26

Which function gives every row a unique sequence number?

- A. RANK()
- B. DENSE_RANK()
- C. ROW_NUMBER()
- D. COUNT()

Q27

For:

100

100

90

what does RANK() produce?

- A. 1,2,3
- B. 1,1,2
- C. 1,1,3
- D. 0,0,1

Q28

For:

100

100

90

what does DENSE_RANK() produce?

- A. 1,2,3
- B. 1,1,2
- C. 1,1,3
- D. 0,0,1

Q29

What does:

PARTITION BY user_id

do?

Q30

What's the difference between:

```
SUM(page_count)
and:
SUM(page_count) OVER (
PARTITION BY user_id
)
?
```

Q31

What does:

```
LAG(page_count) OVER (
ORDER BY id
)
return?
```

Q32

Why can't you solve:

Top 3 documents **per user**

```
with:
ORDER BY page_count DESC
LIMIT 3
?
```

Q33

Which window function is commonly used to keep the latest record per ID?

- A. AVG()**
- B. ROW_NUMBER()**
- C. SUM()**
- D. LEAD()**